

392. Is Subsequence:

Given two strings `s` and `t`, return `true` if `s` is a **subsequence** of `t`, or `false` otherwise.

A **subsequence** of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., `"ace"` is a subsequence of `"a b c d e"` while `"aec"` is not).

Example 1:

```
Input: s = "abc", t = "ahbgdc"
Output: true
```

Example 2:

```
Input: s = "axc", t = "ahbgdc"
Output: false
```

Constraints:

- `0 <= s.length <= 100`
- `0 <= t.length <= 104`
- `s` and `t` consist only of lowercase English letters.

Solution: Time = $O(n)$ and Space = $O(m+n)$

```
class Solution:
    def isSubsequence(self, s: str, t: str) -> bool:
        s = list(s)
        t = list(t)
        #pointers
        l = 0
        r = 0
        ans = True
```

```

while l < len(s):
    if r > len(t)-1:
        return False
    elif s[l] == t[r]:
        l += 1
        r += 1
        if r > len(t):
            ans = False
        ans = True
    elif s[l] != t[r]:
        r += 1
return ans

```

Optimal solution: Time = $O(n)$ and Space = $O(1)$:

```

class Solution:
    def isSubsequence(self, s: str, t: str) -> bool:

        l = 0
        r = 0

        while l < len(s) and r < len(t):

            if s[l] == t[r]:
                l += 1

            r += 1

        return l == len(s)

```